Large Language Models

CISC 7404 - Decision Making

Steven Morad

University of Macau

Admin 2
Language Decision Process 3
Pretraining 6
Supervised Fine Tuning 7
Preferences 9
RLHF 10
Inverse RL 11
Alignment 13
Final Remarks 14
Course Feedback 15

Admin

Admin

Last lecture!

Who started final project coding?

- Who is already training?
- Who learned a good policy?

If you did not make progress, maybe switch to easy backup project

Remember to read the final project requirements carefully!

- Code
- Final report
- Video demonstration

Deadline 3 May (10 days)

Admin

All grades entered except for final project

- Mean grade is 85/100
- Final project worth 30%, still possible to increase grade

Participation scores entered

- Group is 100/100
- Individual

If you reguarly speak and have 0/100 individual, come see me after class

- If you lie, I give you 0/100 group participation
  - "I answered many questions" but I never saw you before

Admin

Quiz 3 graded

- Grades uploaded on moodle
- Mean score 70/100, ignoring missing exams

Some students have 0/100 for quiz 3

- Some already had 100/100 on quiz 1 and 2
- Some students had 0/100 until 14:30 today, issue fixed
  - Double check your score now

Mean over all quiz (quiz 1, quiz 2, quiz 3, drop lowest) = 77/100

Admin

Quiz 3 lowest scoring question: **Q1**

- Makes me sad, I thought this was easy question
- Many students do not know what actor-critic is
  - Q learning/TD learning is **not** actor-critic!
  - Actor critic combines Q/V learning with policy gradient
  - **Question:** Can someone explain why Q1 was difficult?

Admin

Different opinions from **Q1** part 2:

- Math too hard, like coding better (60% respondents)
- Like math, coding too hard (30% respondents)
- Do not like coding or math (change your degree!) (10% respondents)
- Never knew JAX, it is very fast, like learning it (80% respondents)
- Never knew JAX, do not like it (20% respondents)

Admin

Shared opinions

- Do not like quizzes
  - One student: Give less time for quizzes
- Class makes me study a lot
  - Great! This is how you learn material
- Like Q learning more than actor critic
  - Can understand Q learning
  - Cannot understand actor critic
- Assignment 2 stressful
  - After success, feels magical!

Admin

Opinion: Do not like participation, some students are shy

- You must be brave
- I also do not like talking, I worry about making mistakes
- Researcher/engineer/professor **must** be good at presentations
- Good at programming/math **is not enough**
- Many smarter scientists than Yann LeCun or Geoffrey Hinton
  - Backpropagation published twice before Hinton
    - Other 2 researchers have worse social/presentation skills
- It is not fair, but a reality of the world
  - If you want to succeed, you must overcome shyness

Admin

Question for next year: Quiz format?

1. 3 quizzes total, 1 quizzes dropped
2. 2 quizzes total, 0 quizzes dropped
3. 1 final exam instead of quizzes

Question for next year: Homework 2 choice

1. Choose either DQN or PG
2. Complete both DQN and PG

Question for next year: Final project

1. Choose your own problem
2. I give you either Atari or MuJoCo game to solve

Language Decision Process

Language Decision Process

In my deep learning course, we viewed LLMs as deep learning

- Today, we will view LLMs as decision making machines

We will combine everything we learned to create LLMs:

- Actor critic
  - Policy gradient
  - V/Q functions
  - Advantage
- Imitation learning
- Offline RL
- POMDPs

You are smarter than you think, we have everything to build LLMs!

Language Decision Process

Let us consider the problem of language modeling

User gives us variable-length text

We want to reply with different text

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Language Decision Process

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Question: What is the action space A for this task?

Answer 1: All possible text $A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}^n$

Answer 2: All possible words $A =$
$\{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, \texttt{<EOS>}\}$

Language Decision Process

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}^n$$

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, \texttt{<EOS>}\}$$

First approach, produce the entire response at once

Second approach, produce response one word at a time

Question: Which follows MDP definition?

Policy output

Answer: First method, one input produces one output $a \sim \pi(\cdot \mid s; \theta_\pi)$

User input

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}^n$$

Question: Any issues with this action space?

Language Decision Process

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}^n$$

Answer: $n \in [0, \infty]$ so action space is infinite

But policy gradient can represent continuous/infinite action space

Question: Why does policy gradient not work here?

BenBen joint angles are continuous and ordered

Action $a = 1.8\pi$ rad similar to $a = 1.9\pi$ rad, policy can generalize

For LLM action space, meaning of Apple $\neq$ meaning of Aardvark

Policy cannot generalize over infinite discrete actions!

Language Decision Process

$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}^n$ Very difficult to model

Instead, factorize action space into single word or **token**

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, \texttt{<EOS>}\}$$

We implement the action space as a dictionary or integer→token map

$$\{0 : \text{Aardvark},\ 1 : \text{Apple},\ \ldots,\ 128,000 : \text{Zebra}\}$$

Action 1 corresponds to the word "Apple"

This is still a very large action space!

- DeepSeek R1 $|A| = 128,000$

Language Decision Process

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, \texttt{<EOS>}\}$$

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

$a_t \in A$

$a_{t-1} \in A$

$a_{t+1} \in A$

Each token/word in the response is an action

But decision processes require a state for every action

$$a_t \sim \pi(\cdot \mid s_t; \theta_\pi)$$

Language Decision Process

$$A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}\}$$

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

$a_t \in A$

Roses are red, violets are blue, you are beautiful, and I love you

Question: What is s_t? ($a_t \sim \pi(\cdot \mid s_t; \theta_\pi)$)

Answer: Everything we have seen, and everything we have done

Question: Is this a POMDP or MDP? **Answer:** POMDP!

Language Decision Process

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Question: What is our observation space $\mathfrak{O}$?

All possible words $\mathfrak{O} = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, <\text{EOS}>\}$

Same as the action space! $A = \{\text{Aardvark}, \text{Apple}, \ldots, \text{Zebra}, <\text{EOS}>\}$

This simplifies our POMDP state

Language Decision Process

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

$a_t \in A$

Roses are red, violets are blue, you are beautiful, and I love you

$$s_t = \begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix}$$

Question: Can we make the state more compact/efficient?

Language Decision Process

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

$a_t \in A$

Roses are red, violets are blue, you are beautiful, and I love you

$$s_t = \begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix} \qquad s_t = f\left(\begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix}, \theta_f \right) = f(\boldsymbol{\tau}_t, \theta_f)$$

Language Decision Process

$$s_t = \begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix} \qquad s_t = f\left(\begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix}, \theta_f \right) = f(\boldsymbol{\tau}_t, \theta_f)$$

Question: What models can we use for f?

- Transformer (GPT-4, Qwen, DeepSeek R1, LLaMA)
- Recurrent neural network (Mamba)
- Transformer + RNN (Griffin, Google Gemma, Tencent Hunyuan)

Language Decision Process

$$s_t = f\left(\begin{bmatrix} \text{Hello} \\ , \\ \text{please} \\ \vdots \\ , \\ \text{you} \end{bmatrix}, \theta_f\right) = f(\boldsymbol{\tau}_t, \theta_f) \qquad \pi(a \mid s_t; \theta_\pi)$$

f is a very large memory model (99% of parameters)

The policy is the final layer of the LLM (simple linear classifier)

$$\pi(a \mid \boldsymbol{s}_t; \boldsymbol{\theta}_\pi) = \mathrm{softmax}(\boldsymbol{\theta}_\pi^\top \boldsymbol{s}_t) \qquad \boldsymbol{\theta}_\pi \in \mathbb{R}^{|A| \times |S|}$$

Language Decision Process

One more example to make the decision process clear

$$s_t = f(\tau_t, \theta_f)$$

$$a_t \sim \pi(a \mid s_t; \theta_\pi)$$

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Language Decision Process

$$s_t = f(\tau_t, \theta_f)$$

$$a_t \sim \pi(a \mid s_t; \theta_\pi)$$

User: Hello, please write me a poem.

Agent: Sure thing ! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Language Decision Process

$$s_t = f(\tau_t, \theta_f)$$

$$a_t \sim \pi(a \mid s_t; \theta_\pi)$$

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

We understand the policy θ_π and memory θ_f

Now how do we learn θ_π and θ_f?

Pretraining

Pretraining

Now, we understand the decision making process

- Observation space $\mathfrak{O}$
- Action space A
- Policy $\pi(a \mid s; \theta_\pi)$
- Memory $f(\boldsymbol{\tau}_t, \theta_f)$

Question: Do we understand S? **Answer:** No, using POMDP

- S is the latent space learned by θ_f
  - It has some meaning, but humans cannot understand it!

Question: Do we understand Tr? **Answer:** No!

- We do not know the meaning of S, so we cannot know Tr

Pretraining

We covered:

- Observation space $\mathfrak{O}$
- Action space A
- Policy $\pi(a \mid s; \theta_\pi)$
- Memory $f(\boldsymbol{\tau}_t, \theta_f)$
- State space S
- Transition function Tr

Question: What are we missing?

Answer: Reward function $\mathcal{R}$

Pretraining

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Question: What should our reward function $\mathcal{R}$ for this be?

Answer: RLHF?

RLHF will not work yet, GPT-3 requires 45TB of data

RLHF requires humans to annotate 40,000,000,000,000 pages of text

Cannot use RLHF now, maybe later

Pretraining

User: Hello, please write me a poem.

Agent: Sure thing! Here is a poem:

Roses are red, violets are blue, you are beautiful, and I love you

Hard to write reward function for text

Question: What do we do when we cannot find a reward function?

Answer: Imitation learning!

Pretraining

Using behavior cloning, policy can learn to speak like humans

$$\underbrace{\pi(a \mid s; \theta_\pi)}_{\text{Learned policy}} = \underbrace{\pi(a \mid s; \theta_\beta)}_{\text{Human speech}}$$

$$\arg\min_{\theta_\pi} \sum_{s \in \boldsymbol{X}} \sum_{a \in A} -\underbrace{\pi(a \mid s; \theta_\beta)}_{\text{Human speech}} \log \underbrace{\pi(a \mid s; \theta_\pi)}_{\text{Learned policy}}$$

Question: Am I missing anything? Hint: Where does s come from?

Answer: Must learn θ_f to find $s = f(\boldsymbol{\tau}, \theta_f)$

Pretraining

$$\arg\min_{\theta_\pi} \sum_{s \in \boldsymbol{X}} \sum_{a \in A} -\underbrace{\pi(a \mid s; \theta_\beta)}_{\text{Human speech}} \log \underbrace{\pi(a \mid s; \theta_\pi)}_{\text{Learned policy}}$$

$$s_t = f(\boldsymbol{\tau}_t, \theta_f)$$

$o_0, a_0, \ldots, o_t$

Plug in for s and find both θ_π, θ_f

Human thought

$$\underbrace{\arg\min_{\theta_\pi, \theta_f} \sum_{\boldsymbol{\tau} \in \boldsymbol{X}} \sum_{t=0}^{n} \sum_{a \in A} -\pi(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\beta) \log \pi(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\pi)}_{\text{Speak like human}}$$

Model learns to think like human (θ_f) and talk like human (θ_π)

Pretraining

$$\arg\min_{\theta_\pi, \theta_f} \sum_{\boldsymbol{\tau} \in \boldsymbol{X}} \sum_{t=0}^{n} \sum_{a \in A} -\pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\beta\big) \log \pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\pi\big)$$

In my deep learning course, I call this **Generative PreTraining** (GPT)

In this course, I prefer to call it imitation learning!

Key idea: GPT is imitation learning

Objective is to speak like a human (imitate speech)

To speak like a human, model learn to think like a human (imitate thought)

Pretraining

$$\arg\min_{\theta_\pi, \theta_f} \sum_{\boldsymbol{\tau} \in \boldsymbol{X}} \sum_{t=0}^{n} \sum_{a \in A} -\pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\beta\big) \log \pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\pi\big)$$

In BC, we learn from an offline dataset $\boldsymbol{X}$ collected following θ_β

How do we get $\boldsymbol{X}$ for our LLM?

I don't know, this is a tech company secret!

- Crawl websites
- Instagram/Little Red Book comments
- Download books

I know the datasets today are very huge (petabytes!)

Pretraining

$$\arg\min_{\theta_\pi, \theta_f} \sum_{\boldsymbol{\tau} \in \boldsymbol{X}} \sum_{t=0}^{n} \sum_{a \in A} -\pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\beta\big) \log \pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\pi\big)$$

Example: You want to create an LLM startup

1. Convince investors to give you 100B MOP
2. Buy 10,000 GPUs
3. Collect 1 PB dataset
4. Implement deep transformer f and policy π
5. Train BC objective for 3 months

Now you have a very human LLM

- After so much training, the policy knows to think/speak like a human

Pretraining

Time to show your powerful LLM to your investors

User: Hello, please write me a poem.

Agent: No, poems are stupid.

Now your investors are screaming, saying you wasted their money

Question: What happened?

Answer: Our human expert θ_β is a combination many "experts"

- Many of these "experts" are idiots or lazy
- Video comments, internet forums, etc

Pretraining

$$\pi(a_+ \mid s_0; \theta_\beta) = 0.5$$

$$\pi(a_- \mid s_0; \theta_\beta) = 0.5$$

We acted like an internet comment instead of poet

Both poems and comments in our dataset, our model will imitate both

Pretraining

Question: Idiots in the dataset. What can we do?

Answer 1: Delete idiots from dataset (supervised fine tuning)

Answer 2: Offline RL

Supervised Fine Tuning

Supervised Fine Tuning

Deleting idiots from the dataset is **Supervised Fine Tuning** (SFT)

1. Train BC on the large dataset like before
2. Continue training with a **small** and **accurate** dataset

Variant 1: Change how the model thinks and talks

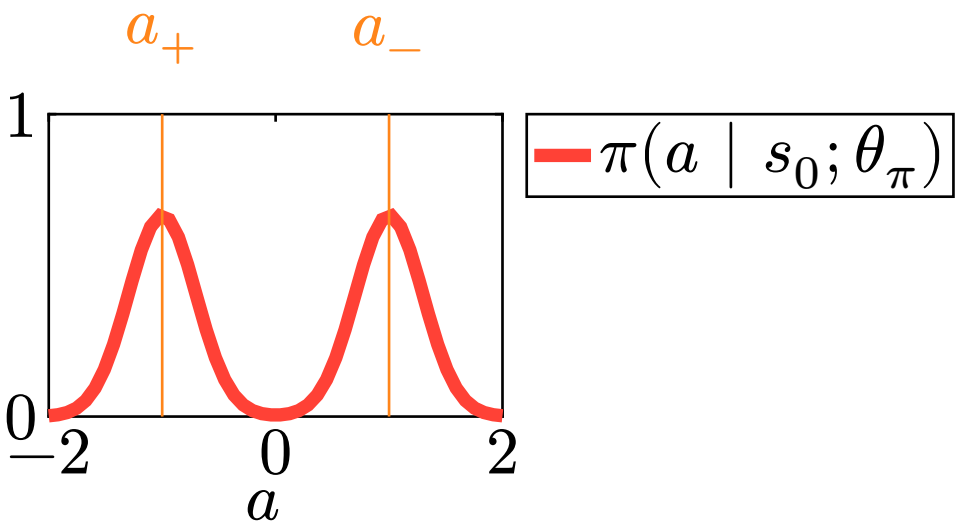
$$\arg\min_{\theta_\pi, \theta_f} \sum_{\boldsymbol{\tau} \in \boldsymbol{X}_{\text{SFT}}} \sum_{t=0}^{n} \sum_{a \in A} -\pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\beta\big) \log \pi\big(a \mid f(\boldsymbol{\tau}_t, \theta_f); \theta_\pi\big)$$

Variant 2: Change how the model talks (θ_f already known)

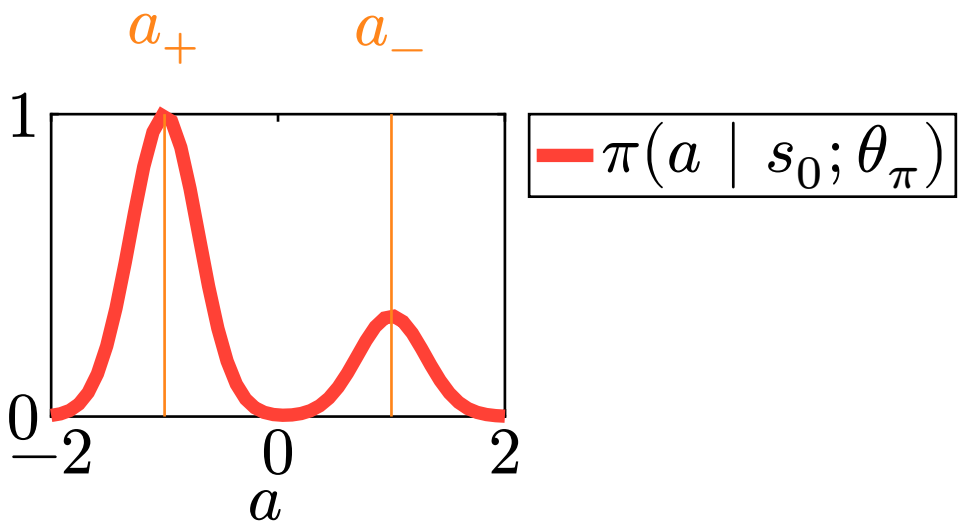
$$\arg\min_{\theta_\pi} \sum_{s \in \boldsymbol{X}} \sum_{a \in A} -\pi\big(a \mid s; \theta_\beta\big) \log \pi\big(a \mid s; \theta_\pi\big)$$

Supervised Fine Tuning

Initially, we model all "experts" from a large dataset. Some experts are bad.



Keep training on small dataset, forget old experts and focus on better experts in small dataset.



Supervised Fine Tuning

SFT works best for **small** and **specific** datasets

- Impersonating anime characters
- Expertise on 18th century painters
- Improve LLM arithmetic

Humans can manually verify the correctness of small datasets

Useful for specific tasks, but does not scale to general intelligence!

- Imitation learning can only imitate humans
- RL can learn optimal policy, better than any human
  - To achieve superhuman intelligence, we must use RL!

Preferences

Preferences

To use RL, we must have rewards

- Use rewards to compute the return (and we maximzie the return)

We already said it is very hard to create reward functions

- With a fixed dataset humans can specify returns/rewards for each datapoint

With rewards, humans must label each subsequence $\boldsymbol{\tau}_t$

$$\mathcal{R}\big(f\big(\text{I}, \theta_f\big)\big), \mathcal{R}\big(f\big(\text{I love}, \theta_f\big)\big), \mathcal{R}\big(f\big(\text{I love you}, \theta_f\big)\big)$$

It is easier for humans to label the return of a trajectory instead

$$\mathcal{G}(\boldsymbol{\tau}) = \mathcal{G}(\text{I love you}) = 12$$

Preferences

Example: I want you to label returns for two trajectories

User: What is Macau?

Agent: Macau is a Special Administrative Region (SAR) in the south of China

Agent: Macau is a city in Asia near the equator

Question: What should the return be for each response?

Humans cannot agree on returns

It is easier for humans to determine **preferences**

Question: Who prefers left response? Right response?

- Many humans will agree with preferences!

Preferences

Problem: Humans provide preferences, but we need return

Can we convert preferences into a return?

Consider two trajectories : τ_+, τ_-

$$\tau_+ = o_0, a_0, o_1, a_1, \ldots \qquad \tau_- = o_0, a_0, o_1, a_1, \ldots$$

$\tau_+ =$ User: What is Macau?
Agent: Macau is a Special Administrative Region (SAR) in the south of China

$\tau_- =$ User: What is Macau?
Agent: Macau is a city in Asia near the equator

Question: How can we design $\mathcal{G}$ that reflects our preference for τ_+?

Answer: Make $\mathcal{G}(\tau_+) > \mathcal{G}(\tau_-)$

Preferences

Measure preference in terms of an unknown return function

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = \underbrace{\mathcal{G}(\boldsymbol{\tau}_+) - \mathcal{G}(\boldsymbol{\tau}_-)}_{\text{Human preference}}$$

Humans bad at choosing scale, rescale to $[0, 1]$ using sigmoid

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = \sigma(\mathcal{G}(\boldsymbol{\tau}_+) - \mathcal{G}(\boldsymbol{\tau}_-))$$

$$\begin{cases} \rho > 0.5 \text{ then } \mathcal{G}(\boldsymbol{\tau}_+) > \mathcal{G}(\boldsymbol{\tau}_-) \\ \rho < 0.5 \text{ then } \mathcal{G}(\boldsymbol{\tau}_+) < \mathcal{G}(\boldsymbol{\tau}_-) \end{cases}$$

Preferences

Definition: The Bradley-Terry advantage relates pairwise preferences to the return, without explicitly computing the return

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = \sigma(\mathcal{G}(\boldsymbol{\tau}_+) - \mathcal{G}(\boldsymbol{\tau}_-))$$

We can treat ρ as the **probability** of a human preferring $\boldsymbol{\tau}_+$ over $\boldsymbol{\tau}_-$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = \frac{\Pr(\text{Human prefers } \boldsymbol{\tau}_+)}{\Pr(\text{Human prefers } \boldsymbol{\tau}_+) + \Pr(\text{Human prefers } \boldsymbol{\tau}_-)}$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = 1 : \text{All humans prefer } \boldsymbol{\tau}_+$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = 0 : \text{All humans prefer } \boldsymbol{\tau}_-$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = 0.8 : \text{Most humans prefer } \boldsymbol{\tau}_+$$

Preferences

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = \sigma(\mathcal{G}(\boldsymbol{\tau}_+) - \mathcal{G}(\boldsymbol{\tau}_-))$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = 1 : \text{All humans prefer } \boldsymbol{\tau}_+$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) = 0 : \text{All humans prefer } \boldsymbol{\tau}_-$$

Can maximize Bradley-Terry advantage to maximize return

$$\arg\max_{\theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_\pi] = \underbrace{\arg\max_{\theta_\pi} \sum_{\boldsymbol{\tau}_-, \boldsymbol{\tau}_+} \rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+)}_{\text{Choose } a \in \boldsymbol{\tau}_+ \text{ not } a \in \boldsymbol{\tau}_-}$$

Now, we have an objective that we can solve with RL!

RLHF

RLHF

With the Bradley-Terry advantage, collect human preference dataset

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{\tau}_- \\ \boldsymbol{\tau}_+ \\ \rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) \end{bmatrix} \qquad \boldsymbol{X} = [\boldsymbol{x}_1 \;\; \boldsymbol{x}_2 \;\; \ldots]$$

Question: How do we use RL on a fixed dataset?

Answer: Offline RL!

RLHF = Offline RL with preferences (human-selected returns)

RLHF

Generative pretraining/imitation learning is 99% of training

- Learn from smart responses mixed with many idiot responses
- Learns general knowledge, human thought and speech

Offline RL/RLHF is 1% of training

- Small but very very important!
- "Unlocks" intelligence by focusing on good thoughts and responses

Offline RL learns to think (θ_f) and speak θ_π **better** than the expert θ_β!

- LLM can become **smarter** than any expert in the dataset
  - Very important! Not possible with imitation learning/GPT

RLHF

Question: What offline RL algorithms do we know?

1. Weighted behavioral cloning with rewards/returns
   - RWR, MARWIL, etc
2. Q learning with constraints
   - BCQ, CQL, etc
   - Constraints prevent overextrapolation
3. Inverse RL
   - Learn reward/return function from dataset, then use online RL

Most people use approaches 1 and 3

Question: Why not approach 2? **Answer:** I don't know, try it out!

RLHF

We name offline RL algorithms differently for RLHF

- Why? So you can "create" a new algorithm and become famous

Weighted behavioral cloning

- Direct Preference Optimization (DPO)
- Kahneman Tversky Optimization (KTO)

Inverse RL

- Constrained REINFORCE/Policy Gradient
- Proximal Policy Optimization (PPO-RLHF)
- Group Relative Policy Optimization (GRPO)

DPO and KTO are very interesting methods, but I have limited time

- Focus on inverse RL

Inverse RL

Inverse RL

In inverse RL, we learn the parameters for the reward function $\theta_{\mathcal{R}}$

$$\mathcal{R}(s, \theta_{\mathcal{R}})$$

In RLHF, it is easier to learn the parameters for return $\theta_{\mathcal{G}}$

$$\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}})$$

Recall our dataset

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{\tau}_- \\ \boldsymbol{\tau}_+ \\ \rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) \end{bmatrix} \qquad \boldsymbol{X} = [\boldsymbol{x}_1 \;\; \boldsymbol{x}_2 \;\; \ldots]$$

Inverse RL

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{\tau}_- \\ \boldsymbol{\tau}_+ \\ \rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+) \end{bmatrix} \qquad \boldsymbol{X} = [\boldsymbol{x}_1 \;\; \boldsymbol{x}_2 \;\; \ldots]$$

$$\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+, \theta_{\mathcal{G}}) = \sigma(\mathcal{G}(\boldsymbol{\tau}_+, \theta_{\mathcal{G}}) - \mathcal{G}(\boldsymbol{\tau}_-, \theta_{\mathcal{G}})) = \Pr(\text{Human prefers } \boldsymbol{\tau}_+)$$

We want to find $\theta_{\mathcal{G}}$ that predicts the human preferences in our dataset

Use cross entropy loss, similar to classification task

$$\theta_{\mathcal{G}} = \arg\max_{\theta_{\mathcal{G}}} \sum_{\boldsymbol{\tau}_+, \boldsymbol{\tau}_- \in \boldsymbol{X}} \underbrace{\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+)}_{\text{Label from } \boldsymbol{X}} \log \underbrace{\sigma(\mathcal{G}(\boldsymbol{\tau}_+, \theta_{\mathcal{G}}) - \mathcal{G}(\boldsymbol{\tau}_-, \theta_{\mathcal{G}}))}_{\text{Learn } \theta_{\mathcal{G}} \text{ to match label}}$$

Inverse RL

$$\theta_{\mathcal{G}} = \arg\max_{\theta_{\mathcal{G}}} \sum_{\boldsymbol{\tau}_+, \boldsymbol{\tau}_- \in \boldsymbol{X}} \underbrace{\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+)}_{\text{Label from } \boldsymbol{X}} \log \underbrace{\sigma(\mathcal{G}(\boldsymbol{\tau}_+, \theta_{\mathcal{G}}) - \mathcal{G}(\boldsymbol{\tau}_-, \theta_{\mathcal{G}}))}_{\text{Learn } \theta_{\mathcal{G}} \text{ to match label}}$$

We often model $\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}})$ using another LLM

$$\mathcal{G}(\boldsymbol{\tau}, \boldsymbol{\theta}_{\mathcal{G}}) = \mathrm{softmax}(\boldsymbol{\theta}_{\mathcal{G}}^{\top} \overset{s}{\boxed{f(\boldsymbol{\tau}, \theta_f)}})$$

We only learn a linear layer $\boldsymbol{\theta}_{\mathcal{G}}$, not the LLM parameters θ_f!

With $\mathcal{G}, \theta_{\mathcal{G}}$, we can compute the return for any trajectory

We use any online RL algorithm to learn θ_π!

Inverse RL

Let us start with REINFORCE (Monte Carlo policy gradient)

$$\theta_{\pi,i+1} = \theta_{\pi,i} + \alpha \cdot \nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_{\pi,i}\right]$$

$$\nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_{\pi,i}\right] = \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_{\pi,i}\right] \cdot \nabla_{\theta_{\pi,i}} \log \pi\left(a_0 \mid s_0; \theta_{\pi,i}\right)$$

Plug in our learned return function $\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}})$

$$\theta_{\pi,i+1} = \theta_{\pi,i} + \alpha \cdot \nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_0; \theta_{\pi,i}\right]$$

$$\nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_0; \theta_{\pi,i}\right] = \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_0; \theta_{\pi,i}\right] \cdot \nabla_{\theta_{\pi,i}} \log \pi\left(a_0 \mid s_0; \theta_{\pi,i}\right)$$

That is it, easy!

Inverse RL

After learning $\theta_{\mathcal{G}}$, we can use normal RL

Standard RL training process:

1. Give the LLM a user query (beginning of $\boldsymbol{\tau}$)
2. LLM generates a response (complete $\boldsymbol{\tau}$)
3. Compute the return $\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}})$
4. Update LLM parameters θ_{π}, θ_{f} using $\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}})$
   - To simplify equations, I will only learn θ_{π}
   - But you can also learn θ_{f}, θ_{π} together

Inverse RL

Definition: Inverse REINFORCE learns a return function using the Bradley-Terry advantage, then learns a policy using REINFORCE

Step 1: Learn the return

$$\theta_{\mathcal{G}} = \arg\max_{\theta_{\mathcal{G}}} \sum_{\boldsymbol{\tau}_+, \boldsymbol{\tau}_- \in \boldsymbol{X}} \underbrace{\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+)}_{\text{Label from } \boldsymbol{X}} \log \underbrace{\sigma(\mathcal{G}(\boldsymbol{\tau}_+, \theta_{\mathcal{G}}) - \mathcal{G}(\boldsymbol{\tau}_-, \theta_{\mathcal{G}}))}_{\text{Learn } \theta_{\mathcal{G}} \text{ to match label}}$$

Step 2: Learn the policy using Monte Carlo policy gradient

$$\theta_{\pi,i+1} = \theta_{\pi,i} + \alpha \cdot \nabla_{\theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_0; \theta_\pi]$$

Inverse RL

Definition: Constrained inverse REINFORCE adds a constraint to ensure the policy does not change too much

Step 1: Learn the return

$$\theta_{\mathcal{G}} = \arg\max_{\theta_{\mathcal{G}}} \sum_{\boldsymbol{\tau}_+, \boldsymbol{\tau}_- \in \boldsymbol{X}} \underbrace{\rho(\boldsymbol{\tau}_-, \boldsymbol{\tau}_+)}_{\text{Label from } \boldsymbol{X}} \log \underbrace{\sigma(\mathcal{G}(\boldsymbol{\tau}_+, \theta_{\mathcal{G}}) - \mathcal{G}(\boldsymbol{\tau}_-, \theta_{\mathcal{G}}))}_{\text{Learn } \theta_{\mathcal{G}} \text{ to match label}}$$

Step 2: Learn the policy with a KL constraint

Policy gradient

$$\theta_{\pi,i+1} = \theta_{\pi,i} + \alpha \cdot \nabla_{\theta_\pi} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_0; \theta_{\pi,i}\right]$$
$$- \nabla_{\theta_{\pi,i+1}} \mathrm{KL}\left[\pi(a \mid s; \theta_\beta), \pi(a \mid s; \theta_{\pi,i+1})\right]$$

KL constraint, stay close to dataset

Inverse RL

Finally, I want to discuss GRPO

GRPO is based on PPO

- PPO is complicated and has too many terms
- I think this will confuse everyone
- The important part of GRPO is group normalization, not PPO

I will present Steven's GRPO: REINFORCE with group normalization

- Much simpler version of GRPO
- Help you understand the key idea behind GRPO

Inverse RL

Definition: Steven's Group Relative Policy Optimization (S-GRPO) normalizes the return using completions of the same user query s_q

$$\boldsymbol{\tau}_i \sim \Pr(\boldsymbol{\tau} \mid s_q; \theta_\pi)$$

$$\theta_{\pi,i+1} = \theta_{\pi,i} + \alpha \cdot \nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) \mid s_q; \theta_{\pi,i}\right]$$

$$\nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}) \mid s_q; \theta_{\pi,i}\right] = \underbrace{\left(\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) - \frac{1}{n}\sum_{i=1}^{n} \mathcal{G}(\boldsymbol{\tau}_i, \theta_{\mathcal{G}})\right)}_{\text{Advantage over other competions}} \nabla_{\theta_{\pi,i}} \log \pi(a \mid s; \theta_{\pi,i})$$

Inverse RL

S-GRPO compare answers to the same query, reducing variance

$$s_q = f(\text{Where is Macau?}, \theta_f)$$

$$\begin{aligned}
\boldsymbol{\tau}_1 &= \text{Macau is a Special Administrative Region (SAR) in } \ldots \\
\boldsymbol{\tau}_2 &= \text{Macau is a city in Asia near the equator } \ldots \\
&\vdots
\end{aligned}$$

$$\nabla_{\theta_{\pi,i}} \mathbb{E}\left[\mathcal{G}(\boldsymbol{\tau}) \mid s_q; \theta_{\pi,i}\right] = \underbrace{\left(\mathcal{G}(\boldsymbol{\tau}, \theta_{\mathcal{G}}) - \frac{1}{n}\sum_{i=1}^{n} \mathcal{G}(\boldsymbol{\tau}_i, \theta_{\mathcal{G}})\right)}_{\text{Advantage over other competions}} \nabla_{\theta_{\pi,i}} \log \pi(a \mid s; \theta_{\pi,i})$$

Similar to Bradley-Terry: Ranking/normalizing trajectories $\boldsymbol{\tau}_-, \boldsymbol{\tau}_+$

Inverse RL

Real GRPO combines group normalization with the PPO objective

- Use the MC return like S-GRPO, deleting V from PPO
  - However, they still use the PPO advantage
- PPO-clip to prevent large policy changes
- Off-policy correction and minibatching
- KL constraint to keep the learned policy near the pretrained policy

Alignment

Alignment

You may hear about LLM **alignment**

Alignment: Does the language model respect human values?

- Polite or rude?
- Unbiased or biased?
- Good or evil?

With RLHF, learn return function from human preferences

- Humans can prefer rude/biased/racist/evil answers
- No ground truth, rude/biased/racist/evil depends on culture
- LLM maximizes the return
  - LLM learns to be biased/racist/evil
  - RL finds superhuman policy, superhuman levels of bias/racism/evil

Alignment

https://www.youtube.com/shorts/RSdIBZX6Adw

https://www.youtube.com/watch?v=eJXDFOwJZMk

Question: What do humans do with less intelligent beings?

Answer: Eat them

Final Remarks

Final Remarks

Throughout the course, I focused on teaching fundamentals and theory

- I know this can be painful
- Some prefer one lecture on PPO, one on SAC, etc instead of basics
- Today, we defined LLMs using fundamentals
  - POMDP
  - Behavioral cloning
  - Offline RL
  - Actor critic
    - Policy gradient
    - V/Q functions

Every complex algorithm is some combination of what you learned

Final Remarks

I am very proud of you all for making it so far!

- From your participation, many of you are already experts!
- It is not expected that you understand everything!
- If you understand MDPs and V/Q functions, that is enough

I know many of you live in Zhuhai, and crossing the border takes time

- I hope these lectures have been useful

And I hope you enjoyed the course!

Final Remarks

The world is a fragile place

- Recent news reminds us how fragile the world is
- We take the ability to study and learn for granted

All of us are born with shortcomings in our reward function

- Greed, bias, fear, indifference
- Consider your own alignment
  - Try and consider the impact of your decisions on others

From assignment 2, you can all create superhuman policies

- You have a powerful tool, you must use it for good
  - Humans are already good at hurting others
  - Do not train superhuman policies to hurt others

Final Remarks

Frodo: I wish the ring had never come to me. I wish none of this had happened.

Gandalf: So do all who live to see such times, but that is not for them to decide. All we have to decide is what to do with the time that is given us.

- We cannot choose our situation s_0
- We cannot control circumstances beyond our control Tr
- The one think we can control, are our actions a

Make sure you choose good actions with the life you are given!

Course Feedback

Course Feedback

Please fill out the course feedback! https://isw.um.edu.mo/siatsl/